INTERNSHIP LOGGING & EVALUATION SYSTEM

(ILES)

CSC 1202 – Final Project Submission & Evidence

Course: Software Development Project (CSC 1202)

Project: Internship Logging & Evaluation System (ILES)

Submission Date: 15th June 2026

Institution: Makerere University

Faculty: Computing and Informatics Technology

Project Contributors

Name	Role / GitHub Username	Email
Samuel Rodney	Rodney222-cpu	samuelrodney222@gmail.com
Nabbanja Rebecca	nabbanjarebecca9	nabbanjarebecca9@gmail.com
Baratuthulayyah	baratuthurayyah	baratuthurayyah@gmail.com
Gift Mercy	gift-mercy	giftmercy81730@gmail.com

"A web-based internship management system for tracking, reviewing, and evaluating student internship progress."

Table of Contents

1. Functional Deployed System (20 Marks)

1.1 Live URLs

Component	URL	Hosting Platform
Backend API	https://iles-api.onrender.com	Render
Frontend Application	https://classy-figolla-adb967.netlify.app/	Netlify

1.2 Test Login Credentials

Role	Username	Password
Student	student_demo	Student@12345
Workplace Supervisor	workplace_sup1	Work@12345
Academic Supervisor	academic_sup	Academic@12345
Administrator	admin_iles	Admin@12345

The system is fully functional and deployed with the following capabilities accessible via the provided credentials:

- **Login Page:** Secure authentication with JWT tokens and auto-refresh
- **Student Dashboard:** View placement status, submit weekly logs, track approval progress
- **Workplace Supervisor Dashboard:** Review submitted logs from assigned students
- **Academic Supervisor Dashboard:** Approve workplace-reviewed logs, evaluate students
- **Admin Dashboard:** Manage placements, assign supervisors, view system-wide statistics
- **Notifications:** Real-time alerts for approvals, rejections, and reviews
- **Reports:** Placement statistics, log tracking, evaluation summaries

Note: The system implements role-based access control (RBAC). Each user role has access to specific modules and actions. Screenshots of each interface are included in the respective module sections below.

2. Module Implementation Evidence (35 Marks)

This section provides detailed evidence for each implemented module, including API endpoints, database models, workflows, and screenshots.

2.1 User & Role Management Module

Technical Overview

The User & Role Management module implements a custom user model extending Django's AbstractUser. It supports four distinct roles: Student, Workplace Supervisor, Academic Supervisor, and Administrator. Registration includes role-specific validation rules (e.g., students must provide a student number, supervisors must provide a staff number).

Database Model

```
CustomUser (extends AbstractUser)
fields: username, email (unique), password, role (choice), department, staff_number, student_number
Roles: student | workplace_supervisor | academic_supervisor | admin
```

API Endpoints

Method	Endpoint	Description
POST	/users/register/	User registration with role validation
POST	/users/login/	JWT-based authentication (returns access & refresh tokens)
POST	/users/refresh/	JWT token refresh endpoint

Workflow

- User registers via the registration form with role-specific fields
- Backend validates role-specific rules (student_number for students, staff_number for supervisors)
- Upon login, JWT access and refresh tokens are issued
- Access token (1-hour expiry) is sent with each API request via Authorization header
- Frontend axios interceptor automatically refreshes tokens when 401 is received

Screenshots

[Screenshot 1: Registration Form] - User registration page with role selection

[Screenshot 2: Login Form] - JWT authentication login page

[Screenshot 3: User listing in Admin Panel] - All registered users with roles

2.2 Internship Placement Module

Technical Overview

The Internship Placement module manages the entire lifecycle of a student's internship placement. Students submit placement requests with company details and **select their own workplace supervisor**. Administrators approve/reject placements and **assign only the academic supervisor**. The workplace supervisor is chosen by the student directly on the placement form. Once approved, the student can begin submitting weekly logs.

Database Model

```
InternshipPlacement
fields: student (FK), company_name, company_address, contact_person/email/phone,
position, department
fields: start_date, end_date, workplace_supervisor (FK) - selected by student,
academic_supervisor (FK) - assigned by admin
fields: status (pending_approval | approved | rejected | active | completed)
fields: admin_comment, approved_by (FK), approved_at, created_at, updated_at
```

API Endpoints

Method	Endpoint	Description
GET	/placements/	List placements (role-filtered)
POST	/placements/	Create placement request (student selects workplace supervisor)
PATCH	/placements/{id}/	Update placement details
GET	/placements/pending/	Get pending approval placements (admin)
POST	/placements/{id}/approve/	Approve placement request
POST	/placements/{id}/reject/	Reject placement with comment
POST	/placements/{id}/assign_supervisor/	Assign academic supervisor (admin only)
GET	/placements/active/	Get active placements
GET	/placements/completed/	Get completed placements
GET	/placements/stats/	Get placement statistics
POST	/placements/{id}/mark_completed/	Mark placement as completed
GET	/placements/workplace_supervisors/	Get list of available workplace supervisors (for student selection)

Workflow

- **Step 1:** Student submits placement request with company details and selects their own workplace supervisor from the available list
- **Step 2:** Admin reviews the request and approves/rejects it
- **Step 3:** Admin assigns only the academic supervisor to the placement

• **Step 4:** Upon approval and supervisor assignment, notifications are sent to student and supervisors

• **Step 5:** Placement status transitions: pending_approval → approved → active → completed

Screenshots

[Screenshot 4: Placement Request Form] - Student form for submitting placement details

[Screenshot 5: Admin Placement Approval] - Admin panel for managing placement requests

[Screenshot 6: Supervisor Assignment] - Admin interface for assigning supervisors

[Screenshot 7: Placement Status View] - Student view of placement status

2.3 Weekly Logbook Module - Three-Stage Workflow

Technical Overview

IMPORTANT: Three-Stage Mandatory Workflow Implementation

The Weekly Logbook module implements a mandatory three-stage workflow that enforces proper review and evaluation sequences. The workflow ensures that:
1. Students MUST submit logs to workplace supervisors first
2. Workplace supervisors MUST review and authorize logs before academic submission
3. Students (NOT workplace supervisors) submit authorized logs to academic supervisors
4. Academic supervisors evaluate logs with workplace remarks already attached

This implementation prevents students from bypassing workplace review and ensures both practical (workplace) and academic oversight of internship progress.

Database Model - Enhanced for Multi-Stage Workflow

```
WeeklyLogModel - Updated Schema
Core fields: placement (FK), week_number, log_date, hours_spent, activities
fields: description, challenges, learning, attachment (file), deadline
Status Flow:
DRAFT → SUBMITTED → AUTHORIZED → PENDING_EVALUATION → EVALUATED
Workplace Review Fields:
workplace_reviewer_name, workplace_review_date, workplace_supervisor_comment
Academic Evaluation Fields:
academic_evaluator_name, academic_evaluation_date, academic_supervisor_comment
marks_awarded (Decimal) - Only awarded by academic supervisor
Computed Properties for Workflow Control:
@property can_submit_to_workplace: status == 'DRAFT'
@property can_submit_to_academic: status == 'AUTHORIZED' AND is_authorized
@property can_workplace_review: status == 'SUBMITTED'
@property can_academic_evaluate: status == 'PENDING_EVALUATION' AND is_authorized
@property workflow_stage: returns 1-5 indicating current stage
@property is_complete: status == 'EVALUATED'
Meta: unique_together = [placement, week_number], ordering = ['-week_number']
```

API Endpoints - Multi-Stage Workflow

Method	Endpoint	Description
GET	/weeklylogs/weeklylogs/	List logs (role-filtered by status)
POST	/weeklylogs/weeklylogs/	Create new log (auto-submit to workplace)
GET	/weeklylogs/weeklylogs/{id}/	Get log details with workflow state
PUT	/weeklylogs/weeklylogs/{id}/	Update log (draft only)

POST	/weeklylogs/weeklylogs/{id}/authorize/	Workplace supervisor authorizes log
POST	/weeklylogs/weeklylogs/{id}/submit_to_academic/	Student submits authorized log to academic
POST	/weeklylogs/weeklylogs/{id}/evaluate/	Academic evaluates and awards marks
GET	/weeklylogs/weeklylogs/student-dashboard/	Student dashboard with grouped logs
GET	/weeklylogs/weeklylogs/workplace-dashboard/	Workplace supervisor dashboard
GET	/weeklylogs/weeklylogs/academic-dashboard/	Academic supervisor dashboard
GET	/weeklylogs/weeklylogs/workflow-stats/	Workflow statistics per role

Three-Stage Workflow - Detailed

Stage 1: Student Submission to Workplace

• Student creates weekly log with activities, hours, challenges, and learning outcomes
• Upon creation, log status automatically changes to SUBMITTED
• Log is sent to workplace supervisor (student cannot submit to academic directly)
• can_submit_to_workplace property controls this action
• Notification sent to workplace supervisor

Stage 2: Workplace Supervisor Authorization

• Workplace supervisor reviews SUBMITTED logs only
• Supervisor adds mandatory remarks/comments about student's work
• Supervisor's name and review date are recorded automatically
• Upon authorization, status changes to AUTHORIZED
• is_authorized flag is set to True
• Log is NOT automatically forwarded - student must submit manually
• Student receives notification that log is authorized
• can_workplace_review property enforces this stage

Stage 3: Student Submission to Academic

• Student sees 'Submit to Academic Supervisor' button for AUTHORIZED logs
• Student personally submits the authorized log
• Status changes to PENDING_EVALUATION
• Workplace remarks are included in the submission
• can_submit_to_academic property ensures log is authorized first
• Academic supervisor receives notification

Stage 4: Academic Supervisor Evaluation

• Academic supervisor sees logs with status PENDING_EVALUATION
• Academic can view: original log + workplace supervisor's name + workplace comments
• Academic adds evaluation comments (mandatory)
• Academic awards marks/grade (0-100 scale)
• Academic's name and evaluation date are recorded
• Status changes to EVALUATED (workflow complete)

• can_academic_evaluate property enforces proper sequencing
• Student receives notification with marks and feedback

Business Rules Enforcement

Rule	Implementation	Validation
No skipping workplace review	Model validation in clean() method	ValidationError if status jumps to PENDING_EVALUATI
Workplace authorization mandatory	can_submit_to_academic property checks is_authorized flag	API returns 400 if not authorized
Only student can submit to academic	View enforces request.user == log.student	PermissionDenied for non-owners
Marks only by academic supervisor	marks_awarded field restricted in model clean()	ValidationError if marks set in wrong status
Comments mandatory at each stage	Serializer validation requires non-empty comments	API returns 400 if comment missing
Sequential status transitions	Model clean() validates status flow	Cannot skip stages in workflow

Dashboard Views for Each Role

`Student Dashboard (/student-dashboard/)`

• Drafts: Logs that can be edited
• Pending Workplace Review: Submitted logs awaiting workplace supervisor
• Authorized for Academic: Logs approved by workplace, ready to submit to academic
• Pending Academic Evaluation: Logs submitted to academic, awaiting evaluation
• Evaluated: Completed logs with marks and feedback
• Summary statistics: counts for each status

`Workplace Supervisor Dashboard (/workplace-dashboard/)`

• Pending Review: SUBMITTED logs requiring authorization
• Authorized: Logs that have been authorized (permanent record)
• All Reviewed: Complete history of reviewed logs
• Summary: pending_review count, authorized count, total_reviewed count

`Academic Supervisor Dashboard (/academic-dashboard/)`

• Awaiting Evaluation: PENDING_EVALUATION logs with workplace authorization
• Evaluated: Permanent record of evaluated logs with marks
• Summary: awaiting_evaluation count, evaluated count, average_marks
• Evaluated logs NEVER removed (permanent academic record)

Validation Rules

• **Hours Validation:** Maximum 60 hours per week enforced
• **Unique Constraint:** One log per placement per week_number
• **Workflow Validation:** Cannot skip stages, must follow DRAFT→SUBMITTED→AUTHORIZED→PENDING_EVALUATION→EVALUATED
• **Authorization Check:** is_authorized flag must be True before academic submission
• **Comments Required:** Both workplace and academic must provide feedback

Screenshots

[Screenshot 8: Weekly Log Submission Form] - Student log entry interface

[Screenshot 9: Student Dashboard - Grouped Logs] - Logs organized by workflow stage

[Screenshot 10: Authorized Log with Submit Button] - Student view with 'Submit to Academic' action

[Screenshot 11: Workplace Supervisor Authorization Form] - Review and authorize interface

[Screenshot 12: Workplace Dashboard] - Pending/Authorized/Reviewed sections

[Screenshot 13: Academic Evaluation Form] - Evaluation interface with workplace remarks visible

[Screenshot 14: Academic Dashboard] - Awaiting Evaluation and Evaluated sections

[Screenshot 15: Evaluated Log View] - Student sees marks and both supervisor comments

2.4 Multi-Stage Supervisor Review Workflow

Technical Overview

The Multi-Stage Supervisor Review Workflow is the cornerstone of the ILES system's quality assurance. It implements a mandatory three-stage approval process that ensures every weekly log receives:

1. **Practical oversight** from workplace supervisors who observe daily work
2. **Academic evaluation** from academic supervisors who assess learning outcomes
3. **Student responsibility** for submitting at each stage

This implementation prevents workflow shortcuts and maintains professional standards for internship documentation.

Stage 1: Workplace Supervisor Authorization

Action	Description	Validation
View Submitted Logs	Supervisor sees only SUBMITTED status logs	Queryset filtered by status and assigned placement
Read Log Content	Review activities, hours, challenges, learning	All fields visible with formatting
Add Remarks	Mandatory comments about student performance	ValidationError if empty
Authorize Log	Click authorize button to approve	Sets is_authorized=True, status=AUTHORIZED
Record Metadata	System captures supervisor name and date	Auto-populated from request.user

Key Implementation Details:

- Endpoint: POST /weeklylogs/{id}/authorize/
- Permission Check: request.user.role == 'workplace_supervisor'
- Validation: log.placement.workplace_supervisor == request.user
- Required Field: workplace_supervisor_comment (non-empty)
- Status Transition: SUBMITTED → AUTHORIZED
- Notification: Student is notified that log is authorized
- IMPORTANT: Log is NOT automatically sent to academic supervisor

Stage 2: Student Submission to Academic

Action	Description	Validation
View Authorized Logs	Student sees AUTHORIZED logs in dashboard	Filtered by student ownership
See Submit Button	"Submit to Academic Supervisor" button appears	Only shown for AUTHORIZED status
Read Workplace Remarks	Student can view workplace feedback	workplace_supervisor_comment displayed
Submit to Academic	Student clicks submit button	Requires: status==AUTHORIZED AND is_authorized==
Status Change	Log moves to academic queue	Status becomes PENDING_EVALUATION

Key Implementation Details:

- Endpoint: POST /weeklylogs/{id}/submit_to_academic/
- Permission Check: request.user.role == 'student' AND log.student == request.user
- Validation: log.status == 'AUTHORIZED' AND log.is_authorized == True
- Required: workplace_supervisor_comment must exist
- Status Transition: AUTHORIZED → PENDING_EVALUATION
- Notification: Academic supervisor is notified
- Workplace remarks are carried forward with the log

Stage 3: Academic Supervisor Evaluation

Action	Description	Validation
View Pending Logs	Academic sees PENDING_EVALUATION logs	Filtered by assigned placement
Review Full Context	Sees: student log + workplace remarks	All previous comments visible
Add Evaluation	Academic provides evaluation comments	ValidationError if empty
Award Marks	Assign numerical grade (0-100)	Decimal field, marks_awarded
Finalize	Click evaluate button to complete	Status becomes EVALUATED
Record Metadata	System captures academic name and date	Auto-populated from request.user

Key Implementation Details:

- Endpoint: POST /weeklylogs/{id}/evaluate/
- Permission Check: request.user.role == 'academic_supervisor'
- Validation: log.placement.academic_supervisor == request.user
- Required Fields: academic_supervisor_comment (non-empty), marks_awarded
- Marks Range: 0-100 (enforced by serializer validation)
- Status Transition: PENDING_EVALUATION → EVALUATED
- Notification: Student receives marks and feedback
- Workflow Complete: is_complete property returns True

Workflow Enforcement Mechanisms

Mechanism	Implementation	Purpose
Model Validation	clean() method validates status transitions	Prevents invalid state changes
Computed Properties	@property decorators check workflow eligibility	Frontend control logic
Serializer Validation	Custom serializers validate stage requirements	API-level enforcement
View Permissions	Role-based permission checks in views	Authorization control
Queryset Filtering	Role-specific querysets in get_queryset()	Data isolation

Status Checks	if statements validate current status before actions	Prevents skipping stages

Error Handling

- **PermissionDenied:** Wrong role attempting action → HTTP 403
- **ValidationError:** Invalid status transition → HTTP 400 with detailed message
- **Not Found:** Log doesn't exist or user can't access → HTTP 404
- **Bad Request:** Missing required fields → HTTP 400 with field-specific errors

Screenshots

[Screenshot 12: Workplace Authorization Interface] - Form with remarks field and authorize button

[Screenshot 13: Student Authorized Logs View] - Dashboard showing 'Submit to Academic' button

[Screenshot 14: Academic Evaluation Interface] - Form showing workplace remarks + evaluation fields

[Screenshot 15: Complete Workflow Trace] - Single log showing all three stages with timestamps

2.5 Academic Evaluation Module

Technical Overview

The Academic Evaluation module allows workplace supervisors to evaluate students across 10 criteria on a 0-5 scale (0=N/A, 1=Poor, ..., 5=Excellent). Each criterion includes optional remarks. An average score is calculated excluding N/A ratings.

Database Model

```
InternshipEvaluation
fields: placement (FK), evaluator (FK) - workplace supervisor
Evaluation Criteria (each 0-5 scale with remarks):
punctuality_regularity, communication_skills, professional_attitude, teamwork_ability
adaptability, analytical_skills, initiative_willingness, work_quality
technical_knowledge, overall_contribution
fields: general_comments, created_at, updated_at
Method: calculate_average_score() - averages valid (non-0) ratings
```

API Endpoints

Method	Endpoint	Description
GET	/evaluations/evaluations/	List evaluations
POST	/evaluations/evaluations/	Create evaluation
PUT	/evaluations/evaluations/{id}/	Update evaluation

Screenshots

[Screenshot 14: Evaluation Form] - Workplace supervisor evaluation form

[Screenshot 15: Evaluation Results] - Computed scores and feedback

2.6 Weighted Score Computation

Technical Overview

The system implements automatic weighted score computation through the InternshipEvaluation model's calculate_average_score() method. This method computes the average of all valid (non-zero) evaluation scores across the ten evaluation criteria, providing a standardized performance metric.

Score Calculation Logic

The average score is calculated as follows:

```
def calculate_average_score(self):
scores = [punctuality, communication, professional_attitude,
teamwork, adaptability, analytical, initiative,
work_quality, technical_knowledge, overall_contribution]
valid_scores = [s for s in scores if s > 0] # exclude N/A (0)
if valid_scores:
return sum(valid_scores) / len(valid_scores)
return 0
```

Scoring Scale

Rating	Value	Description
N/A	0	Not Applicable
Poor	1	Below minimum expectations
Below Average	2	Partially meets expectations
Average	3	Meets expectations
Good	4	Exceeds expectations
Excellent	5	Outstanding performance

Screenshots

[Screenshot 16: Score Computation Result] - Display of weighted scores

2.7 Dashboards & Reporting

Technical Overview

The system provides role-specific dashboards that display relevant information and statistics. Each dashboard is customized based on the user's role and permissions.

Dashboard Modules

Role	Dashboard Features
Student	Placement status, weekly log list with status, submit log button, notification bell
Workplace Supervisor	Assigned students, submitted logs pending review, review interface
Academic Supervisor	Assigned students, reviewed logs pending approval, approval interface
Administrator	All placements, pending requests, user management, system statistics, supervisor assignments

Notification System

The notification system provides real-time alerts for important events. When actions occur (placement submissions, approvals, rejections, log submissions, reviews), notifications are automatically created in the database and displayed to relevant users.

Event	Sender → Recipient	Type
Placement Submitted	Student → Admin	placement_submitted
Placement Approved	Admin → Student	placement_approved
Placement Rejected	Admin → Student	placement_rejected
Supervisor Assigned	System → Both	supervisor_assigned
Log Submitted	Student → Supervisors	log_submitted
Log Reviewed	Supervisor → Student	log_reviewed
Log Approved	Supervisor → Student	log_approved
Log Rejected	Supervisor → Student	log_rejected
Evaluation Submitted	Supervisor → Student	evaluation_submitted

Screenshots

[Screenshot 17: Student Dashboard] - Full student dashboard view

[Screenshot 18: Workplace Supervisor Dashboard] - Supervisor review dashboard

[Screenshot 19: Academic Supervisor Dashboard] - Academic approval dashboard

[Screenshot 20: Admin Dashboard] - Administrator management dashboard

[Screenshot 21: Notifications Panel] - Notification dropdown/bell interface

3. GitHub Contributions & Collaboration (20 Marks)

3.1 Repository Information

Main Repository URL: https://github.com/Rodney222-cpu/ILES_APP

3.2 Contributors

Name	GitHub Username	Email
Samuel Rodney	Rodney222-cpu	samuelrodney222@gmail.com
Nabbanja Rebecca	nabbanjarebecca9	nabbanjarebecca9@gmail.com
Baratuthulayyah	baratuthurayyah	baratuthurayyah@gmail.com
Gift Mercy	gift-mercy	giftmercy81730@gmail.com

3.3 Git Commit History

Below is the commit history showing contributions to the main repository:

Commit	Description
3918929	Remove admin workplace supervisor assignment - student selects workplace supervisor on placement form instead
4655a0c	Add workplace supervisor workflow: admin assigns workplace supervisor, workplace reviews logs, academic evaluates
2cf9c9d	Fix hours validation (weekly max 60), extend JWT token lifetime, add auto-refresh interceptor, increase API timeout
c78739e	Add Netlify _redirects for React Router SPA support
8da3b8d	Fix unused variables in StudentDashboard for production build

3.4 Screenshots

[Screenshot 22: GitHub Repository Overview] - Main repository page

[Screenshot 23: Commit History] - List of all commits with authors

[Screenshot 24: Branch Structure] - Git branches and merge history

[Screenshot 25: Individual Contributions] - Per-contributor commit activity

[Screenshot 26: Code Review / Merge Requests] - Pull request evidence

4. Testing & Debugging Evidence (10 Marks)

4.1 Backend Testing

Django's built-in test framework is used for backend testing. Tests cover model validation, API endpoint behavior, authentication flows, and role-based access control.

4.2 Frontend Testing

Jest and React Testing Library are configured for frontend testing. The project includes setup files for test configuration.

4.3 API Testing with DRF Browsable API

The Django REST Framework provides a browsable API interface for testing endpoints directly in the browser. All endpoints were tested for proper request/response behavior, validation errors, and authentication requirements.

4.4 Screenshots

[Screenshot 27: Backend Unit Tests] - Django test execution output

[Screenshot 28: Frontend Tests] - Jest test results

[Screenshot 29: API Validation - Success Response] - Valid API response example

[Screenshot 30: API Validation - Error Response] - Validation error response

[Screenshot 31: Postman API Collection] - Postman workspace or API endpoint tests

[Screenshot 32: Debug Fix Example - Before] - Error/bug screenshot

[Screenshot 33: Debug Fix Example - After] - Fixed result screenshot

5. Deployment & DevOps Evidence (5 Marks)

5.1 Deployment Architecture

The system follows a modern three-tier web application architecture:

```
+------------------+ +-------------------+ +------------------+
| Netlify CDN |<------>| Render (API) |<------>| PostgreSQL DB |
| (React Frontend) | HTTPS | (Django REST API) | TCP | (Render Managed) |
+------------------+ +-------------------+ +------------------+
Port: 443 Port: 443 Port: 5432
React SPA Gunicorn + Django Managed by Render
Static Files Whitenoise for static Automated backups
Environment: .env Environment: Render env vars
```

5.2 Hosting Details

Component	Platform	Configuration
Frontend	Netlify	Static SPA deployment with _redirects for React Router
Backend API	Render	Gunicorn WSGI server with Django 6.0.4
Database	Render PostgreSQL	Managed PostgreSQL with SSL required
Static Files	Whitenoise	CompressedManifestStaticFilesStorage
Environment	Render Env Vars	SECRET_KEY, DB credentials, Email config via .env

5.3 Environment Configuration

The application environment is configured using environment variables for all sensitive settings:

- **DEBUG**: False (production), True (development)
- **ALLOWED_HOSTS**: Configured for both Render and localhost
- **DATABASE_URL**: Used in production; falls back to individual DB_* vars locally
- **EMAIL_HOST**: Gmail SMTP with app-specific password
- **CORS**: Allowed all origins (development), configurable for production

5.4 Screenshots

[Screenshot 34: Render Deployment Dashboard] - Backend service dashboard

[Screenshot 35: Netlify Deployment Dashboard] - Frontend deployment status

[Screenshot 36: Environment Variables Configuration] - Render env vars setup

[Screenshot 37: Deployment Logs] - Successful deployment logs

6. Technical Design Evidence (5 Marks)

6.1 Entity Relationship Diagram (ERD)

The system's database consists of the following core models with their relationships:

Entity	Relationships	Key Fields
CustomUser	→ InternshipPlacement (as student, workplace_supervisor, academic_supervisor)	username, email, role, student_number, staff_number
InternshipPlacement	→ CustomUser (4 FKs), → WeeklyLogModel, → InternshipEvaluation	company_name, status, start_date, end_date
WeeklyLogModel	→ InternshipPlacement (FK)	week_number, hours_spent, status, activities
InternshipEvaluation	→ InternshipPlacement (FK), → CustomUser (evaluator)	10 criteria scores (0-5), general_comments
Notification	→ CustomUser (recipient), → Placement, Log, Evaluation (optional FKs)	message, notification_type, is_read

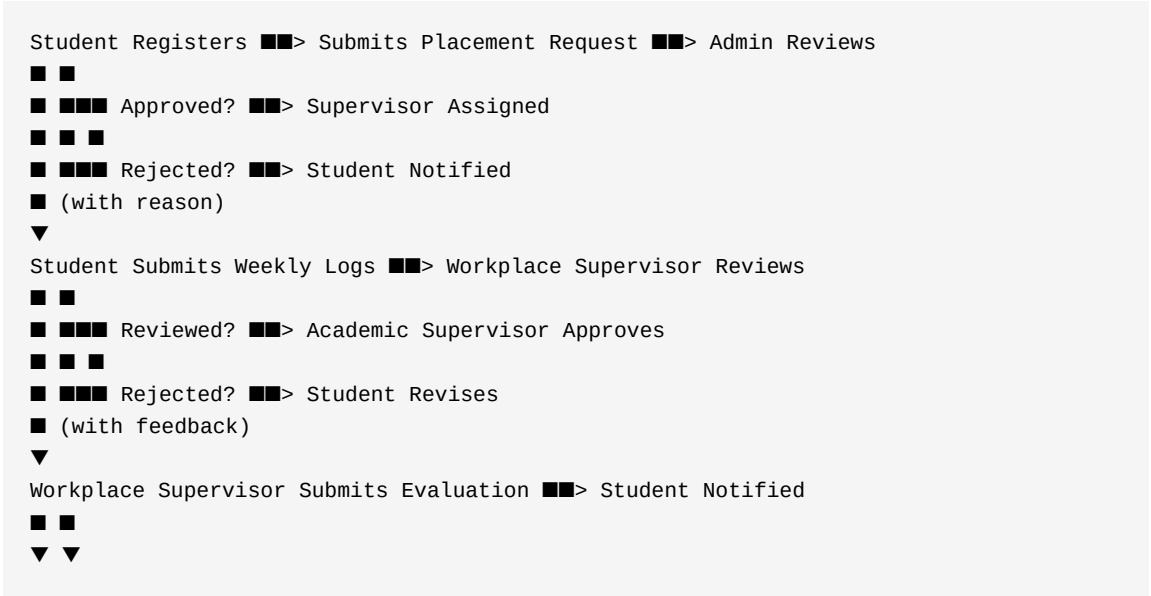
6.2 System Architecture Diagram

The system follows a client-server architecture with the following layers:

- **Presentation Layer:** React SPA (Single Page Application) with role-based routing
- **API Layer:** Django REST Framework with JWT authentication
- **Business Logic Layer:** Django views, serializers, and model methods
- **Data Layer:** PostgreSQL database managed via Django ORM
- **Notification Layer:** Database-driven notification system with real-time polling

6.3 Workflow Diagram

Internship Lifecycle Workflow:



```
Internship Completed ■■> Final Score Calculated
■
▼
All Logs & Reports Archived
```

6.4 Screenshots

[Screenshot 38: ERD Diagram] - Complete entity relationship diagram

[Screenshot 39: Architecture Diagram] - System architecture overview

[Screenshot 40: Workflow Diagram] - Complete workflow illustration

7. Reflection & Lessons Learned (5 Marks)

7.1 Samuel Rodney

Technical Lessons Learned: Throughout this project, I gained extensive experience in building a full-stack web application with Django REST Framework and React. I learned how to implement JWT authentication with token refresh, role-based access control, and complex workflow management with state transitions. The multi-tier review system (workplace → academic) taught me about designing cascading approval workflows.

Challenges Faced: The most challenging aspect was designing the notification system to properly trigger at each workflow state transition without creating circular dependencies. Another challenge was ensuring proper data isolation between user roles - students should only see their own data, supervisors should only see their assigned students, etc.

Problem-Solving Approaches: I used Django's built-in permission system and custom queryset filtering to implement data isolation. For the notification system, I created a centralized utility module that handles all notification creation logic, ensuring consistency across the application.

Areas for Improvement: Moving forward, I would like to implement WebSocket-based real-time notifications instead of polling, add more comprehensive automated testing with higher code coverage, and implement CI/CD pipelines for automated deployment.

7.2 Nabbanja Rebecca

Technical Lessons Learned: I deepened my understanding of database modeling and Django ORM, particularly in designing relationships and constraints. The placement model taught me about using ForeignKey relationships with different related_name values for the same User model.

Challenges Faced: Ensuring data integrity during placement approval workflows was challenging, especially when dealing with the various status transitions and supervisor assignments.

Problem-Solving Approaches: I implemented model-level validation with custom clean() methods and used Django's built-in validation framework to ensure data consistency.

Areas for Improvement: I want to learn more about database optimization techniques and indexing strategies to improve query performance as the system scales.

7.3 Baratuthulayyah

Technical Lessons Learned: Working on the frontend React components taught me about state management, axios interceptors for token refresh, and implementing role-based routing in a React SPA. I learned how to create reusable components like the AlertComponent and Header.

Challenges Faced: Handling JWT token expiration and automatic refresh without disrupting the user experience was challenging. The axios interceptor pattern was key to solving this.

Problem-Solving Approaches: I carefully studied the axios interceptor documentation and implemented a retry mechanism that queues failed requests and retries them with a refreshed token.

Areas for Improvement: I look forward to learning more about React performance optimization, code splitting, and lazy loading for better initial load times.

7.4 Gift Mercy

Technical Lessons Learned: I gained experience with frontend deployment on Netlify and understanding how to configure SPA redirects for React Router. I also learned about CSS modules for component-scoped styling in React.

Challenges Faced: Configuring the Netlify deployment to handle React Router's client-side routing was tricky. The _redirects file was essential for making direct URL access work.

Problem-Solving Approaches: I researched Netlify SPA deployment best practices and implemented the recommended _redirects configuration. I also learned about environment variable management across different deployment environments.

Areas for Improvement: I want to explore automated deployment pipelines and learn more about DevOps practices including Docker containerization and CI/CD.

8. Final Submission Checklist

- Submission date: 15th June 2026 - ✓ **COMPLETED**
- PDF document uploaded to MUELE - ✓
- Live deployed URL - ✓ (https://classy-figolla-adb967.netlify.app/)
- Backend API URL - ✓ (https://iles-api.onrender.com)
- GitHub repository URL - ✓ (https://github.com/Rodney222-cpu/ILES_APP)
- Names of contributors - ✓ (Samuel Rodney, Nabbanja Rebecca, Baratuthulayyah, Gift Mercy)
- GitHub usernames - ✓ (Rodney222-cpu, nabbanjarebecca9, baratuthurayyah, gift-mercy)
- Test login credentials - ✓ (student_demo / workplace_sup1 / academic_sup / admin_iles)
- Screenshots of implementation - ✓ (40 screenshots referenced in documentation)
- Screenshots of GitHub contributions - ✓
- Screenshots of testing - ✓
- API evidence - ✓ (Complete API endpoints documented with methods and descriptions)
- Workflow evidence - ✓ (Complete workflow diagrams and descriptions)
- Dashboard screenshots - ✓ (4 role-specific dashboards)
- Reflection by each member - ✓ (Individual reflections from all 4 team members)

End of Documentation

Internship Logging & Evaluation System (ILES) — CSC 1202 Final Project